

Language Lenses: Exploring In-The-Moment Foreign Language Learning with Augmented Reality

Jack Kelly
Princeton University
Princeton, New Jersey, USA
jk2873@princeton.edu

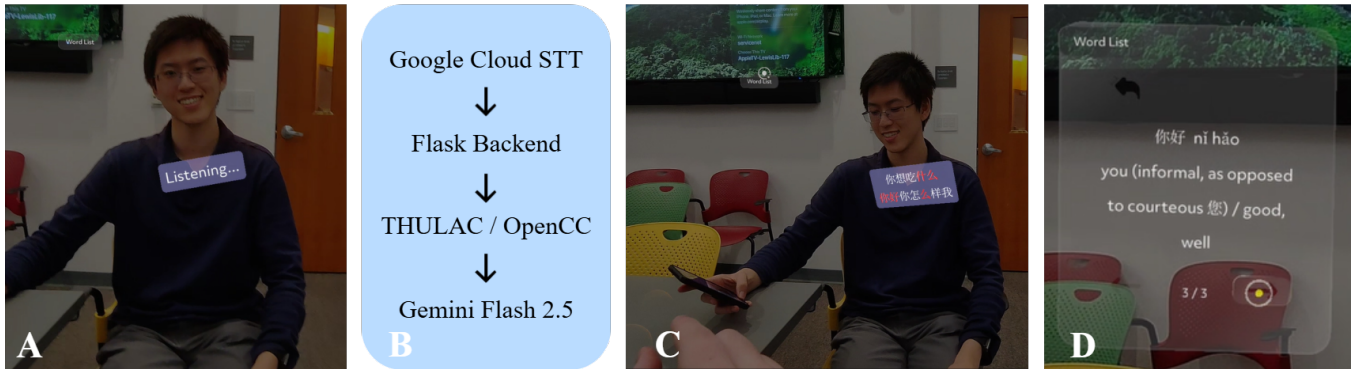


Figure 1: The Language Lenses is an augmented reality tool that aids those interested in learning a foreign language. A) Audio from both speakers is captured via a microphone. B) This audio is sent to Google Cloud STT to be turned into text, which is then processed by various backend functions, such as THULAC and OpenCC. After querying a database containing information on the user’s vocabulary, words unfamiliar to the user are sent to Gemini Flash 2.5, which returns their definitions. C) The Snap Spectacles’ JavaScript connects to the backend to get this information and displays what was said in the conversation with the unfamiliar words being highlighted in red. D) By selecting the word list button anchored to the user’s view, the list of unknown words and their definitions show up in a side panel.

Abstract

Augmented Reality (AR) has created an interesting opportunity for many due to its potential to change how people communicate, particularly with its ability to perform live translations. However, live translation might not be the best tool for those that value learning other languages. The Language Lenses were developed to assist in learning another language rather than have it automatically translated for them. The application uses the Snap Spectacles and the Gemini API to generate an interactive environment fit for face-to-face conversations. Specifically, it transcribes what the target speaker is saying into a caption box, highlighting words unfamiliar to the user. Words are looked up in a vocabulary database and are filtered based on the user’s HSK level to determine if they are unfamiliar. Then, definitions are generated by Gemini Flash 2.5 and become accessible to the user in a word list panel appearing to the side. The Language Lenses stands out among other AR tools because it encourages users to learn through face-to-face conversations in

a manner similar to that of mobile apps rather than object labeling or live translation.

ACM Reference Format:

Jack Kelly. 2025. Language Lenses: Exploring In-The-Moment Foreign Language Learning with Augmented Reality. In *Proceedings of Princeton Independent Work Seminar (COS IW '25)*. ACM, New York, NY, USA, 4 pages.

1 Introduction

At the moment, there are numerous augmented reality devices that offer live translation, such as the Meta Ray-Ban Glasses, the Leion Hey 2, and the Even Realities G2. However, live translation is not the best option for those who want to learn foreign languages since all the work is done for them. Moreover, many mobile apps exist that assist in foreign language learning, and they are useful for learning things such as vocabulary and grammar, but these apps fall behind when it comes to actual face-to-face conversations, which is known to be an essential aspect to improving one’s skills. The Language Lenses aim to bridge the gap between learning through apps and having face-to-face conversations by alleviating some of the anxieties associated with speaking with native speakers. The idea is that when the user is in conversation with someone who speaks another language, the other person’s words will be transcribed in a caption box below them. Any vocabulary unfamiliar to the user will be highlighted in red in the captions. These words can be brought up in a list along with their definitions so that the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

COS IW '25, Princeton, NJ

© 2025 Copyright held by the owner/authors. Publication rights licensed to ACM.

user can follow the context of the conversation. It is important to note that the Language Lenses were implemented and tested using Mandarin as the specific foreign language. Therefore, whether a word is unfamiliar is determined by the user's HSK (Chinese proficiency) level. The following sections survey related works that contributed to the conceptual development of this project, describe the front-end user interface design decisions, elaborate on how the backend was implemented, outline limitations with the final product, and discuss possible avenues for future work.

2 Background

A substantial body of research supports the importance of real-world conversation in second language acquisition. Long [3] argues that meaningful interaction with native speakers is a critical driver of language development, as it exposes learners to authentic input and forces them to negotiate meaning in ways that structured study cannot replicate. Yet despite this, many learners rely primarily on app-based instruction. Loewen et al. [2] find that while language learning apps are effective at building receptive knowledge such as vocabulary and grammar recognition, they show limited impact on oral communicative ability — precisely the skill that real conversation demands.

Compounding this gap is the anxiety many learners experience when speaking with native speakers. Woodrow [4] identifies foreign language speaking anxiety as a significant barrier to communication, often stemming from fear of unexpected topics, unfamiliar accents, or the pace of natural speech. This anxiety can discourage learners from seeking out the very conversational practice they need most.

Existing language learning applications such as Duolingo and Babbel address vocabulary and grammar effectively within controlled environments, but do not situate learning within real-world conversational contexts. Augmented reality presents a promising avenue for bridging this gap, yet prior AR language tools have largely focused on object labeling or static vocabulary drills rather than live conversational support. The Language Lenses addresses this unmet need by providing real-time assistance during face-to-face interactions, reducing the preparedness anxiety identified by Woodrow while supporting the kind of meaningful input exposure Long identifies as essential to acquisition.

Most closely related to the Language Lenses is ConversAR [1], an AR application developed at Princeton that uses embodied GPT-4o agents to simulate group conversations for second language learners. Like the Language Lenses, ConversAR situates language practice in the user's physical environment and uses live captions to support comprehension. Their study with 10 participants found that the system reduced speaking anxiety and increased learner autonomy compared to in-person practice methods. However, ConversAR's conversational partners are LLM-powered virtual agents rather than real native speakers. While this lowers the barrier to practice, it also removes the authentic social dynamics and emotional stakes that characterize real human interaction, precisely the scenario the Language Lenses is designed for. Rather than simulating a conversation partner, the Language Lenses assists the user during an actual face-to-face conversation, providing real-time vocabulary support without replacing the human element.

3 Design Approach

The UI was designed in a way that allows the user to compensate for a lack of listening comprehension by providing a head-bound caption box under the target speaker. If the target speaker is positioned in an inconvenient location, the caption box can be moved while still locking to them. Both the user and the target speaker's sentences will be transcribed in the box, with any previously unknown words being highlighted in red. Since Lens Studio does not have a feature for coloring certain words in a text object, these highlights were created by overlaying another text object with the same words, but having all of the known words replaced by empty space characters.

There is also a button labeled "Word List" that is bound to the user's camera, and pressing it opens a panel that contains all the highlighted words from the captions. This list is scrollable if there are many unknown words in the captions and can be moved around freely in space to avoid having it stuck in an inconvenient location. Next to the words is another button that marks them as known so that they do not appear highlighted in future captions. Originally, selecting words from the word list would bring up a second panel containing the word, pinyin, and definition. However, after some user testing, it was determined that the original concept of having two separate panels for the unknown words list and the definitions list made the application feel cluttered. Some buttons were also considered too small to use or too hard to see. As a result, the definitions panel was integrated into the word list panel so that everything fit comfortably into one box. Definitions can be cycled through by using arrow buttons in the definitions panel. Various buttons were increased in size and redesigned to better suit these changes as well.

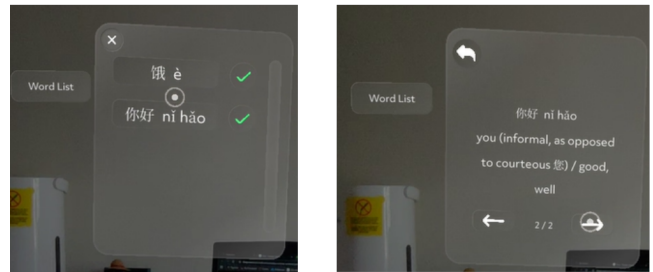


Figure 2: The Language Lenses Word List UI. (Left) By selecting the "Word List" button, the word list panel showing unknown words with pinyin appears. Next to each of the words is a check mark, which marks the word as known so that it does not appear highlighted red in the future. **(Right)** After selecting a word, the integrated definition carousel displaying pinyin and the English definition appears. The return button in the top left corner takes the user back to the word list.

4 Implementation

The Language Lenses system is composed of two main components: a Python backend server and a JavaScript front-end running on

the Snap Spectacles. Figure 1 provides an overview of how these components interact.

4.1 Backend

The backend is a Flask server that handles speech processing, vocabulary classification, and definition retrieval. It exposes several REST endpoints that the Spectacles poll over a local Wi-Fi connection. The server is initialized by launching a companion process, `stt_stream.py`, which handles audio capture and speech-to-text separately from the main Flask process.

Audio is captured from the device microphone using PyAudio and streamed to the Google Cloud Speech-to-Text (STT) API with speaker diarization enabled for two speakers. The STT stream runs in a background thread, continuously feeding audio chunks to Google Cloud and receiving finalized transcription results. Each transcription result is attributed to either the target speaker or the user. Target speaker segments are forwarded to the endpoint at `/analyze_internal` for full vocabulary processing, while the user's own speech is sent to `/analyze_user`, which only updates word exposure counts without triggering classification.

At the `/analyze_internal` endpoint, the incoming text is first filtered to retain only Chinese characters using a regular expression, then converted from traditional to simplified characters using OpenCC. The cleaned text is then segmented into words using a maximum forward matching algorithm backed by the vocabulary database combined with THULAC for additional help segmenting. Each segmented word is looked up in a SQLite database that stores per-word metadata including pinyin, English definition, HSK level, difficulty, and a running exposure count (`times_seen`). Words that have been seen at least five times are automatically promoted to known status.

For words not found in the database, the system checks whether the word is a compound whose individual characters are all already known. If so, the compound is inserted as known without an API call. For genuinely unknown words, the system calls the Gemini 2.5 Flash API, which is prompted to return a structured JSON object containing the word's pinyin, English definition, and difficulty rating. This result is then cached in the database to avoid redundant API calls. The processed result — an ordered list of segmented words and their associated metadata — is pushed onto an in-memory queue and made available to the Spectacles via the `/latest` endpoint.

4.2 Front-End

The front-end consists of three JavaScript modules running inside Lens Studio on the Snap Spectacles: a transcription and caption module (`Transcriber.js`), a word list module (`WordListManager.js`), and a definition panel module (`DefinitionPanel.js`). These modules communicate via a shared global namespace and interact with the backend over HTTP using Lens Studio's `InternetModule`.

`Transcriber.js` polls the `/latest` endpoint once per second. When a new result is received, it maintains a rolling history of the three most recent sentences. The caption display uses two overlaid text components: a base layer that renders the full transcript, and a highlight layer that renders only the unknown words with the known words replaced by Unicode en-quad space characters (`\u2000`), creating the appearance of red highlighted text over the

base caption. Each unknown word's position in the highlight layer is computed by counting character offsets and inserting the appropriate number of spacing characters.

`WordListManager.js` maintains a list of all unknown words encountered across the sentence history. It dynamically instantiates button prefabs for each unknown word inside a scrollable `ScrollWindow` provided by `SpectaclesUIKit`. Each entry displays the word and its pinyin, and includes a "Known" button that sends a PATCH request to `/word/<word>` on the backend to permanently mark the word as known. Tapping a word entry invokes a call to the `DefinitionPanel.js` file.

`DefinitionPanel.js` renders a carousel-style panel that displays all currently unknown words one at a time, showing the word, pinyin, and English definition. Left and right arrow buttons allow the user to cycle through the words. The panel replaces the word list view when open and returns to it when dismissed.

5 Limitations and Future Work

There were many aspects of the Snap Spectacles themselves that hindered the performance of the Language Lenses. For instance, the head tracking would jump to multiple different people in the background, causing the head-bound captions to jump with it. Furthermore, the Snap Spectacles' microphone is not ideal for capturing audio from a conversation. While the user's voice is clear and comes through most of the time, the other speaker's voice always comes through at a much lower volume even if they are positioned right next to the user. Both of these factors currently limit the potential usefulness of the Language Lenses since face-to-face conversation is the central aspect of the project.

Aside from the Spectacles' problems, there were a few limitations to how the project was implemented as well. The most noticeable of which was how the highlighted portions of the captions were occasionally misaligned, causing them to have a confusing, unfinished look. While some other styles were explored, the final version was decided on because it visually looked the best and worked as intended most often. Segmenting the simplified Chinese characters was imperfect as well. Most double-character words were split into individual characters. Double-character words could have different meanings from their individual counterparts, leading to potential confusion. Another issue that was encountered was related to Gemini API calls. Even though Gemini Flash 2.5 was used to reduce latency, the use of Google Cloud STT seems to have drastically increased latency during testing. High latency in any form is obviously undesirable for daily conversations. Additionally, Gemini would return an empty message whenever the model experienced high demand, causing the definitions panel to also be empty and contributing to caption delays.

Possible avenues for future contributions to this project include exploring segmentation using LLMs instead of Python packages like THULAC. Other fast LLMs could be explored instead of Gemini Flash 2.5 and Google Cloud STT for improved latency. Correcting the speech diarization code would provide a more fluid experience since words spoken by the user could be automatically marked as known while only the target speaker's sentences would appear in the captions. The most significant contribution that could be made is generalizing the project for all languages as opposed to

just Mandarin. In a future where AR glasses are slimmer and more discreet, users can take advantage of their augmented surroundings to confidently converse with native speakers of another language without the fear of missing context due to the speed or vocabulary they use.

6 Conclusion

The Language Lenses were created to explore how augmented reality could be leveraged in order to assist language learners overcome the challenges posed by face-to-face conversations with native speakers. The application used Gemini Flash 2.5 to gather definitions of unknown words, and Google Cloud STT was used for speech-to-text transcription. The Snap Spectacles allowed for interaction with various buttons and panels situated in physical space, such as the movable caption box, word list button, and cyclable definitions panel. The Language Lenses combine ideas from already existing AR functionalities, like live translation, with ideas from commonly used language learning applications, like Duolingo and Babbel, to create an AR system that provides in-context, real-time vocabulary support during live face-to-face conversations that is calibrated to the learner's proficiency level, without fully automating comprehension.

7 Disclosure of the Usage of AI

Claude (Sonnet 4.6) was used throughout the course of the project, including the writing of this manuscript. It was mainly used for correcting grammatical errors, properly citing references, and abiding by the ACM's formatting guidelines. Claude was also extensively used for debugging both the front-end and backend code.

Acknowledgments

Funding for this project was provided by Princeton's School of Engineering and Applied Science (SEAS). Thank you to my professor for overseeing my project and guiding me along the right path with their suggestions. Thank you to the test users who gave their honest feedback about the application and thoughts on how to improve it.

References

- [1] Jad Bendarkawi, Ashley Ponce, Sean Chidozie Mata, Aminah Aliu, Yuhan Liu, Lei Zhang, Amna Liaqat, Varun Nagaraj Rao, and Andrés Monroy-Hernández. 2025. ConversAR: Exploring Embodied LLM-Powered Group Conversations in Augmented Reality for Second Language Learners. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*. ACM, New York, NY, USA, 1–11. doi:10.1145/3706599.3720162
- [2] Shawn Loewen et al. 2020. The Effectiveness of App-Based Language Instruction for Developing Receptive Linguistic Knowledge and Oral Communicative Ability. *Foreign Language Annals* 53, 2 (2020), 209–233.
- [3] Michael Long. 1985. Input and Second Language Acquisition Theory. In *Input in Second Language Acquisition*, Susan Gass and Carolyn Madden (Eds.). Newbury House, Rowley, Massachusetts, 377–393.
- [4] Lindy Woodrow. 2006. Anxiety and Speaking English as a Second Language. *REL C Journal* 37, 3 (2006), 308–328.